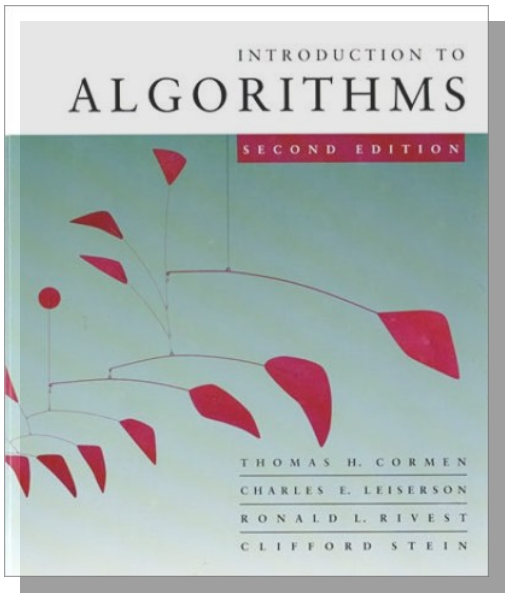
Introduction to Algorithms



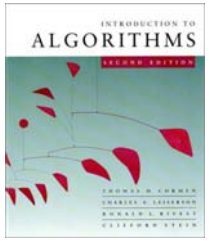
LECTURE 1

Analysis of Algorithms

- Insertion sort
- Asymptotic analysis
- Merge sort
- Recurrences

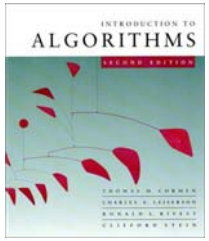
Instructors: Zhaoyi Jiang

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Course information

1. Staff
2. Prerequisites
3. Lectures
4. Handouts
5. Textbook
6. Problem sets
7. Describing algorithms
8. Grading policy



What are algorithms?

- ***definition***

  Any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.

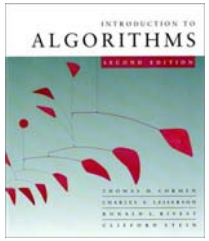
  A sequence of computational steps that transform the input into the output.

- ***Content***

  Data stucture - A way to store and organize data to facilitate access and modifications

  Algorithm analysis - predicting the resources (time and space)

  Algorithm design - Strategy to develop



The problem of sorting

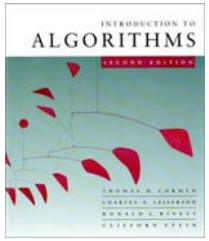
Input: sequence $\langle a_1, a_2, \ldots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \le a'_2 \le \cdots \le a'_n$.

Example:

Input: 8 2 4 9 3 6

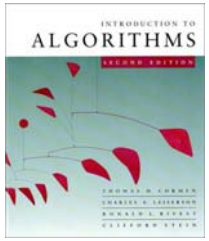
Output: 2 3 4 6 8 9



Insertion sort

"pseudocode"

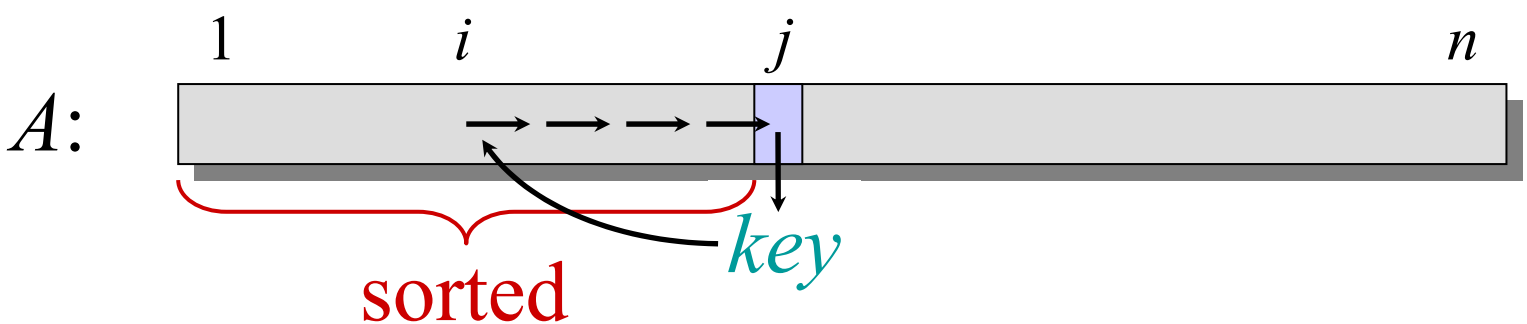
```
INSERTION-SORT (A)
    for j = 2 to A.length
        key = A[j]
        // Insert A[j] into the sorted sequence A[1.. j-1]
        i = j – 1
        while i > 0 and A[i] > key
            A[i+1] = A[i]
            i = i – 1
        A[i+1] = key
```



Insertion sort

"pseudocode"

```
INSERTION-SORT (A)
    for j = 2 to A.length
        key = A[j]
        // Insert A[j] into the sorted sequence A[1.. j-1]
        i = j – 1
        while i > 0 and A[i] > key
            A[i+1] = A[i]
            i = i – 1
        A[i+1] = key
```



Example of insertion sort

8 2 4 9 3 6

Example of insertion sort

8 2 4 9 3 6

Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

Example of insertion sort

8 2 4 9 3 6

2 8 4 9 3 6

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6
2	3	4	8	9	6

Example of insertion sort

8	2	4	9	3	6
2	8	4	9	3	6
2	4	8	9	3	6
2	4	8	9	3	6
2	3	4	8	9	6

Example of insertion sort

8	2	4	9	3	6	
2	8	4	9	3	6	
2	4	8	9	3	6	
2	4	8	9	3	6	
2	3	4	8	9	6	
2	3	4	6	8	9	*done*

Running time

- The running time depends on the input: an already sorted sequence is easier to sort.
- Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.
- Generally, we seek upper bounds on the running time, because everybody likes a guarantee.

INSERTION-SORT(A)	*cost*	*times*
for $j = 2$ **to** $A.length$	c_1	n
$\quad key = A[j]$	c_2	$n-1$
$\quad$ // Insert $A[j]$ into the sorted sequence $A[1..j-1]$.	0	$n-1$
$\quad i = j - 1$	c_4	$n-1$
$\quad$ **while** $i > 0$ and $A[i] > key$	c_5	$\sum_{j=2}^{n} t_j$
$\quad\quad A[i+1] = A[i]$	c_6	$\sum_{j=2}^{n} (t_j - 1)$
$\quad\quad i = i - 1$	c_7	$\sum_{j=2}^{n} (t_j - 1)$
$\quad A[i+1] = key$	c_8	$n-1$

Let $T(n)$ = running time of INSERTION-SORT.

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1)$$

$$+ c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1) .$$

Best case: The array is already sorted. (t_j=1)

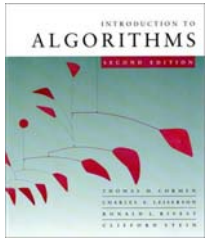
Running time is

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\
&= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \; .
\end{aligned}
$$

Worst case: The array is in reverse sorted order. (t_j=j)

Running time is

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\
&\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\
&= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n \\
&\quad - (c_2 + c_4 + c_5 + c_8) \; .
\end{aligned}
$$



Kinds of analyses

Worst-case: (usually)

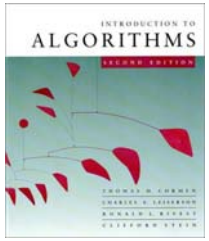
- $T(n)$ = maximum time of algorithm on any input of size n.

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n.
- Need assumption of statistical distribution of inputs.

Best-case: (bogus)

- Cheat with a slow algorithm that works fast on *some* input.



Machine-independent time

What is insertion sort's worst-case time?

- It depends on the speed of our computer:
  - relative speed (on the same machine),
  - absolute speed (on different machines).

BIG IDEA:

- Ignore machine-dependent constants.
- Look at ***growth*** of $T(n)$ as $n \to \infty$.

"Asymptotic Analysis"

Θ-notation

Math:

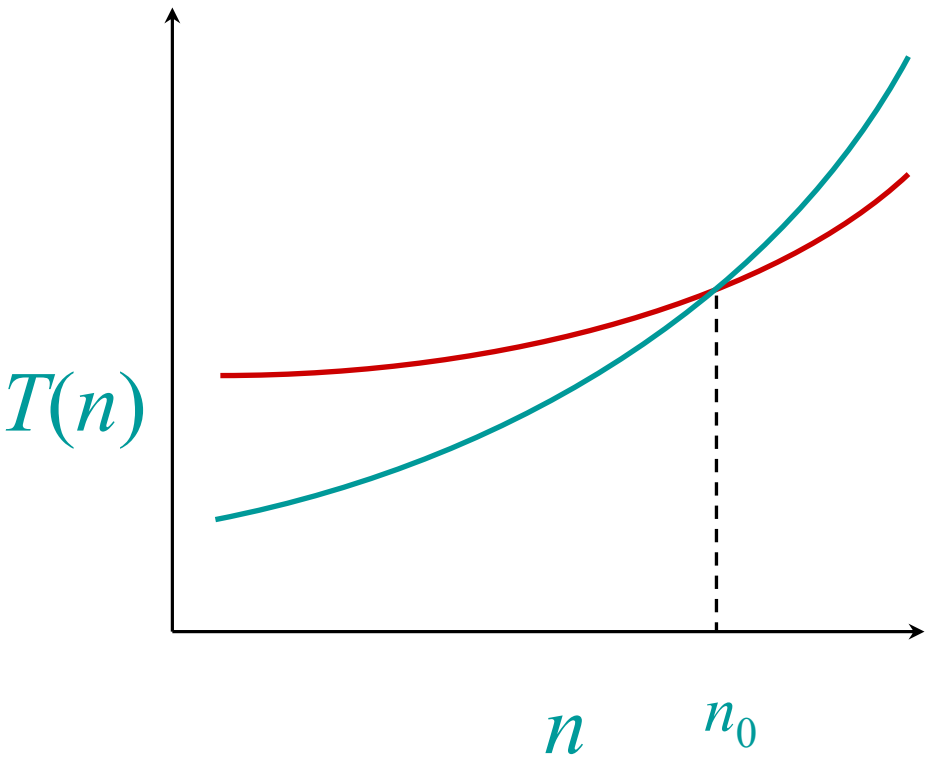
$\Theta(g(n)) = \{ f(n) :$ there exist positive constants c_1, c_2, and n_0 such that $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0 \}$

Engineering:

- Drop low-order terms; ignore leading constants.
- Example: $3n^3 + 90n^2 - 5n + 6046 = \Theta(n^3)$

Asymptotic performance

When n gets large enough, a $\Theta(n^2)$ algorithm *always* beats a $\Theta(n^3)$ algorithm.



- We shouldn't ignore asymptotically slower algorithms, however.
- Real-world design situations often call for a careful balancing of engineering objectives.
- Asymptotic analysis is a useful tool to help to structure our thinking.

Insertion sort analysis

Worst case: Input reverse sorted.

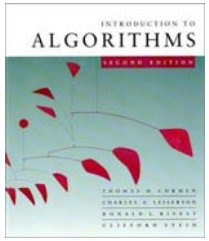
$$T(n) = \sum_{j=2}^{n} \Theta(j) = \Theta(n^2) \qquad \text{[arithmetic series]}$$

Average case: All permutations equally likely.

$$T(n) = \sum_{j=2}^{n} \Theta(j/2) = \Theta(n^2)$$

Is insertion sort a fast sorting algorithm?

- Moderately so, for small n.
- Not at all, for large n.



Merge sort

MERGE-SORT $A[1 \ldots n]$

1. If $n = 1$, done.
2. Recursively sort $A[1 \ldots \lceil n/2 \rceil]$ and $A[\lceil n/2 \rceil + 1 \ldots n]$.
3. "***Merge***" the 2 sorted lists.

Key subroutine: **MERGE**

Merging two sorted arrays

20	12
13	11
7	9
2	1

Merging two sorted arrays

20	12
13	11
7	9
2	1

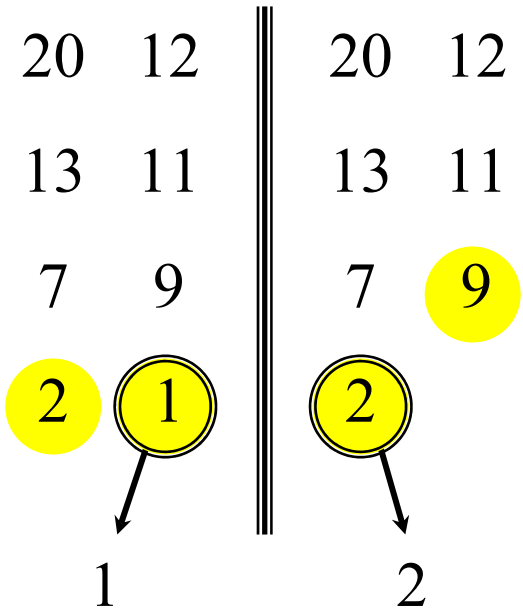
1

Merging two sorted arrays

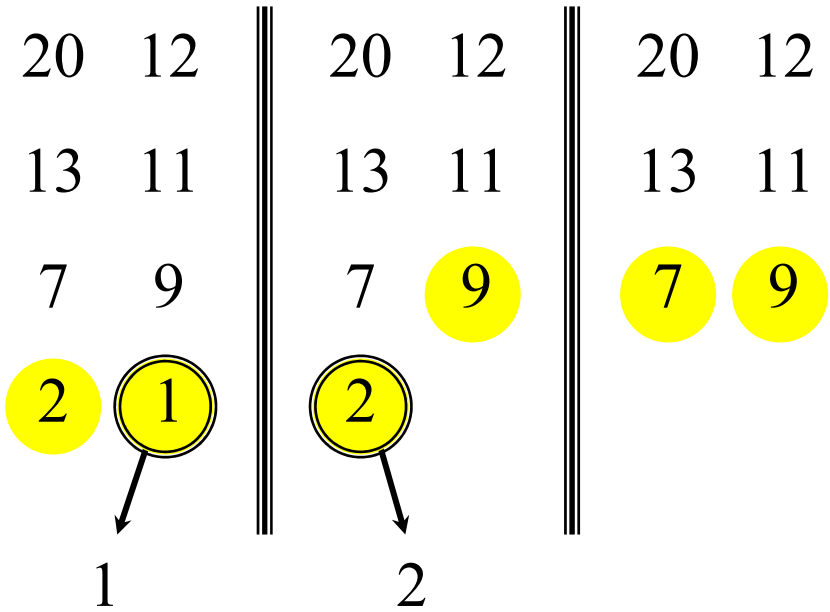
20	12	20	12
13	11	13	11
7	9	7	9
2	1	2	

1

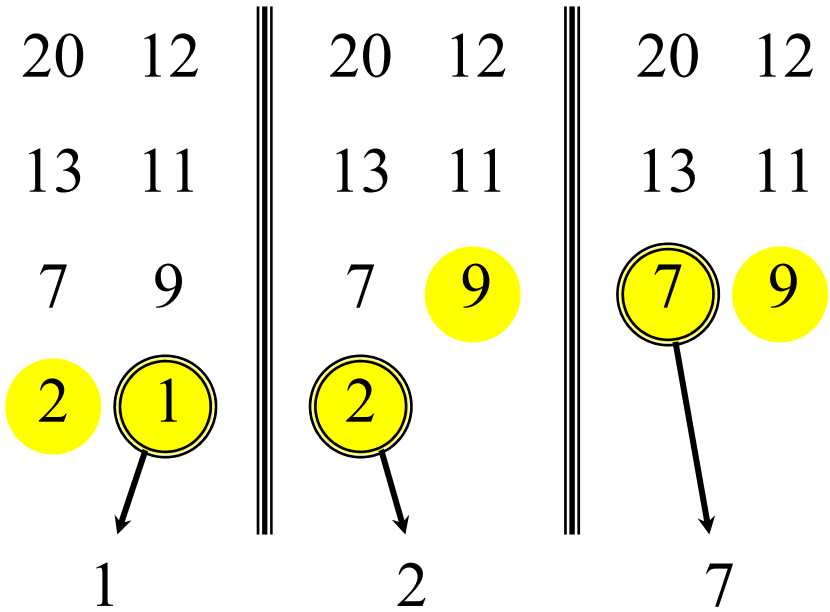
Merging two sorted arrays



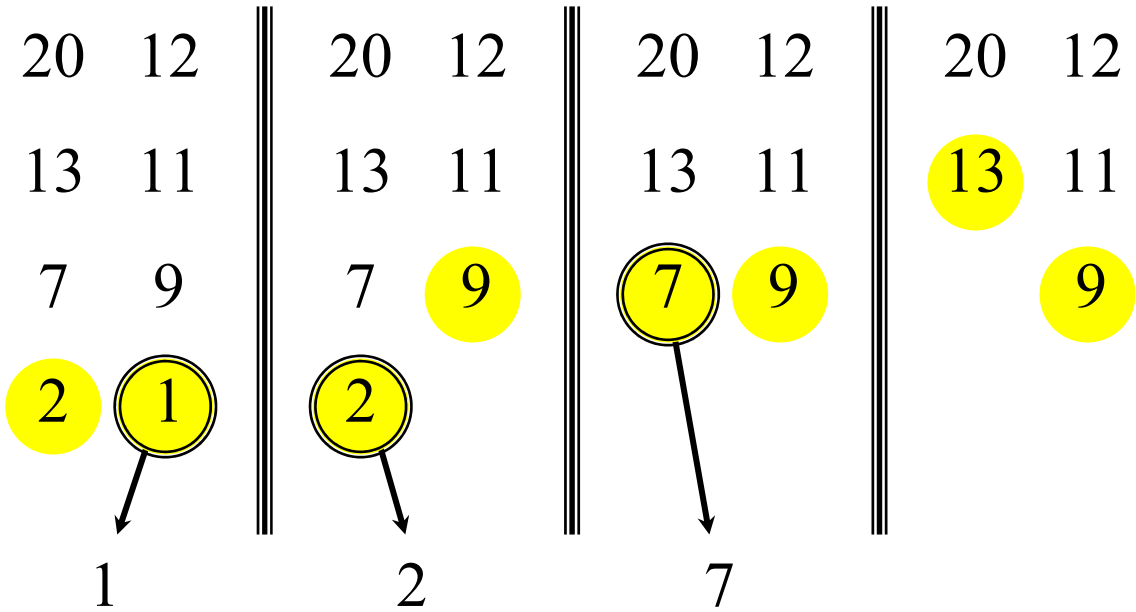
Merging two sorted arrays



Merging two sorted arrays

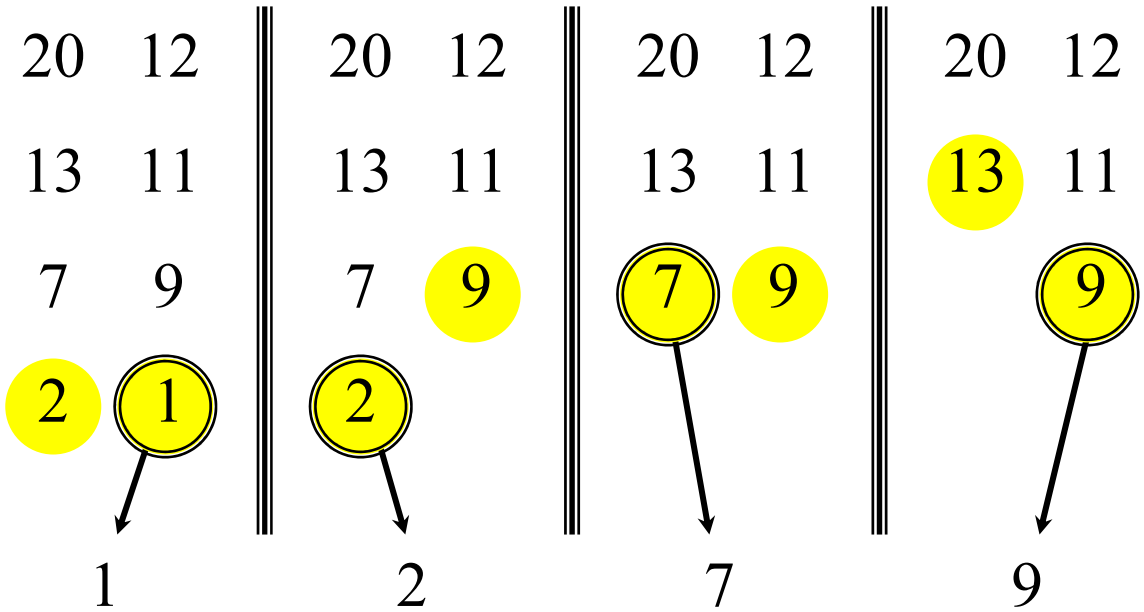


Merging two sorted arrays



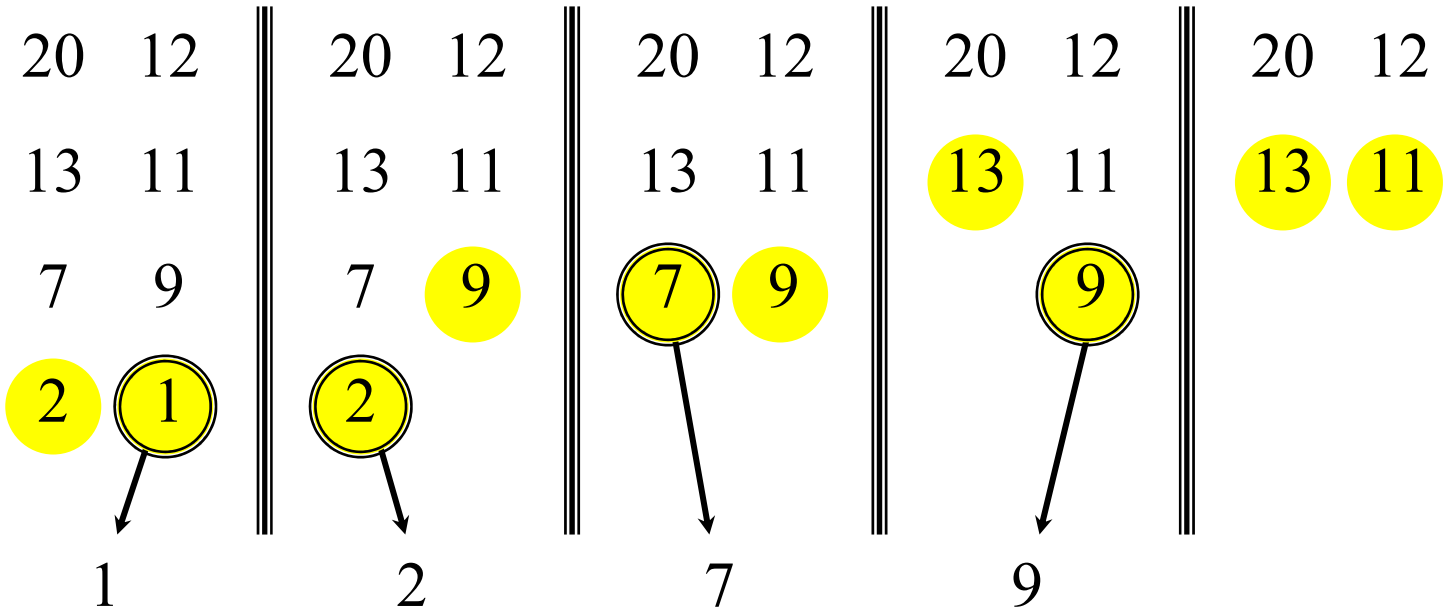
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



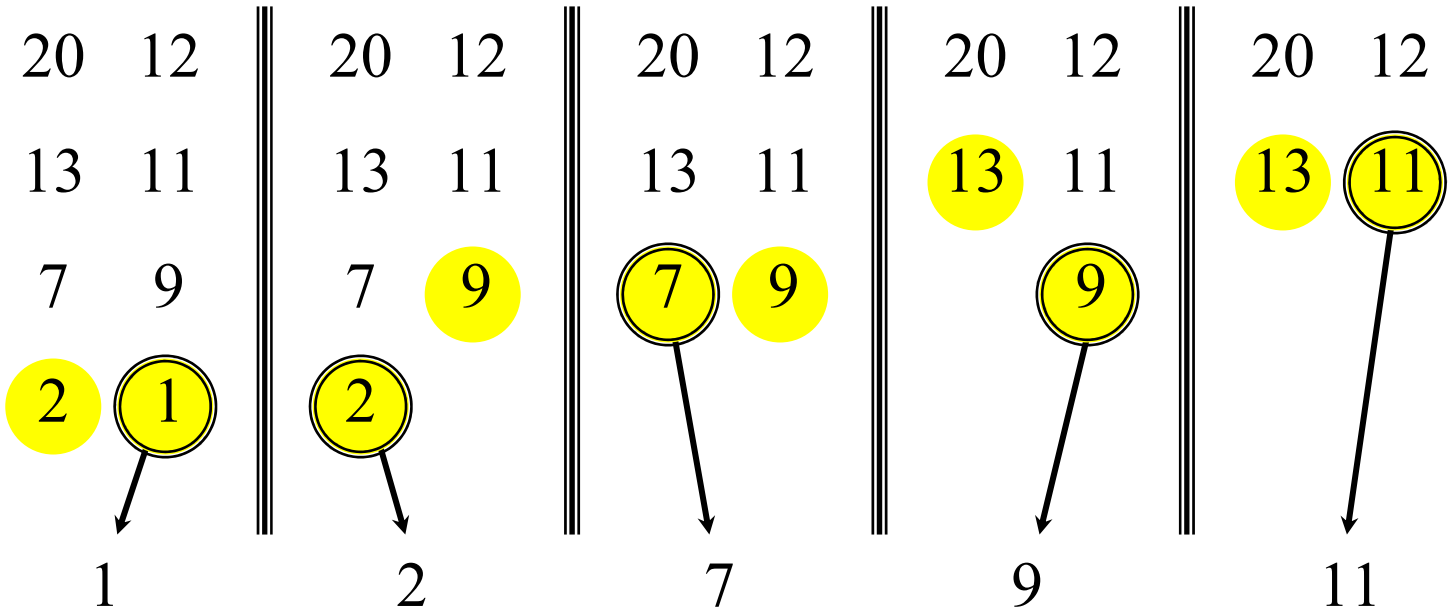
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



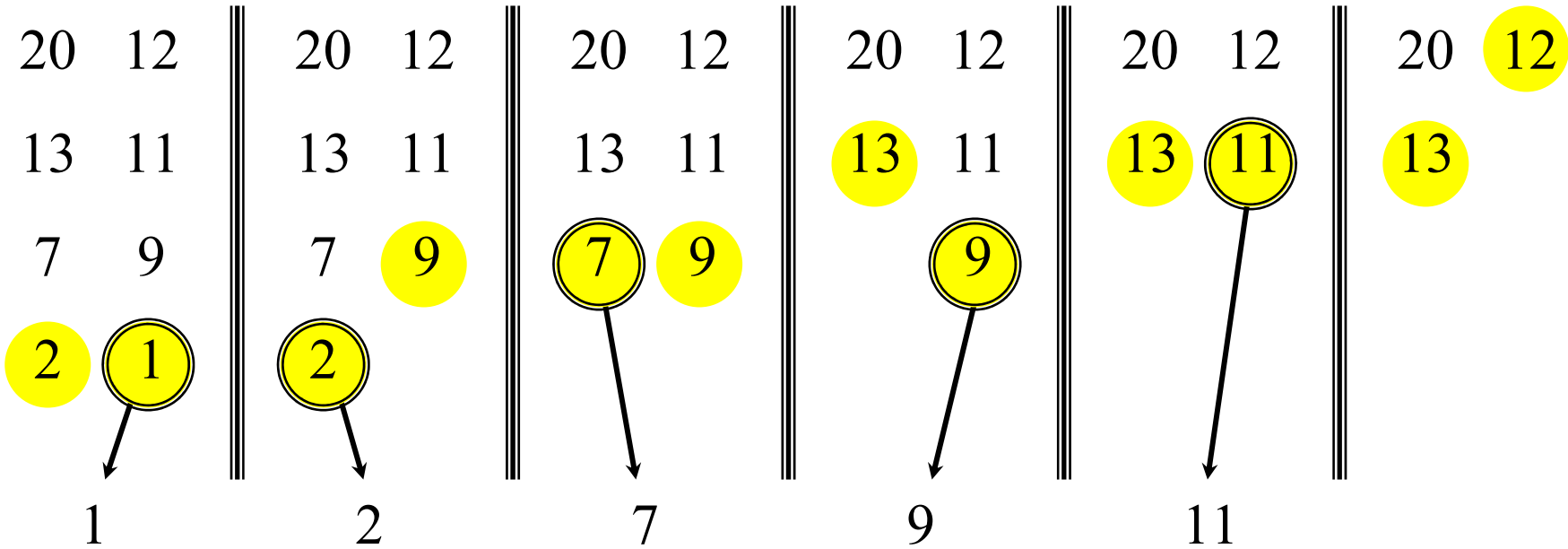
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



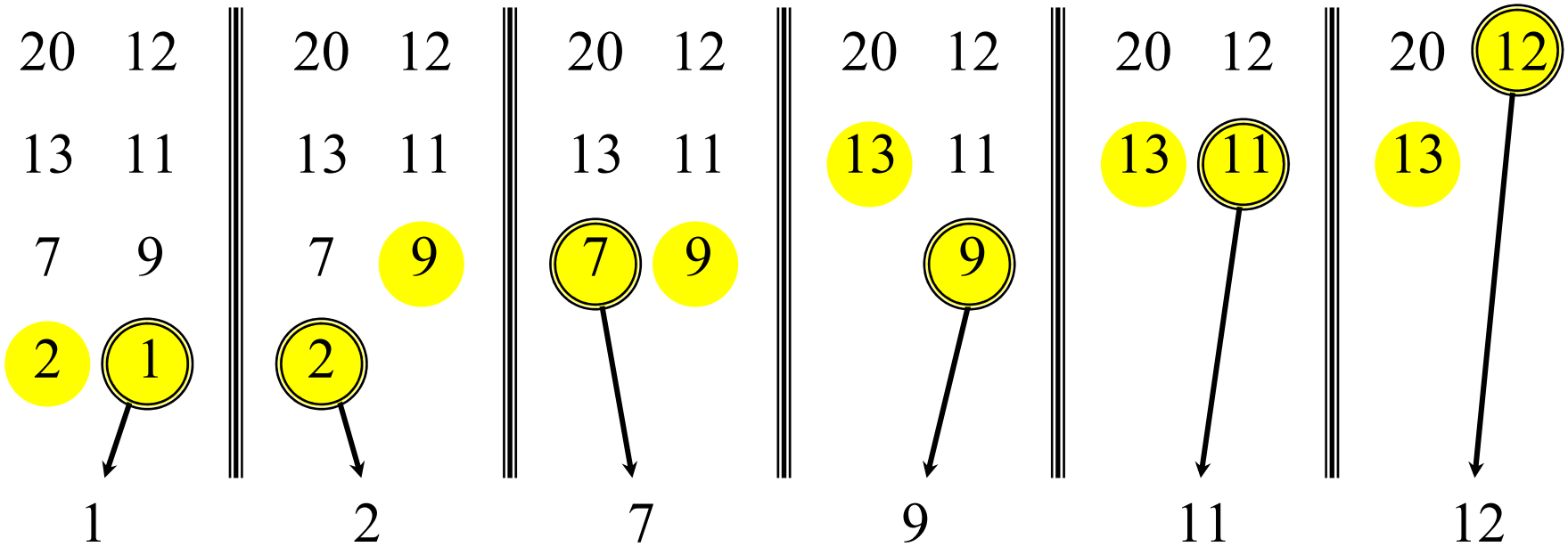
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



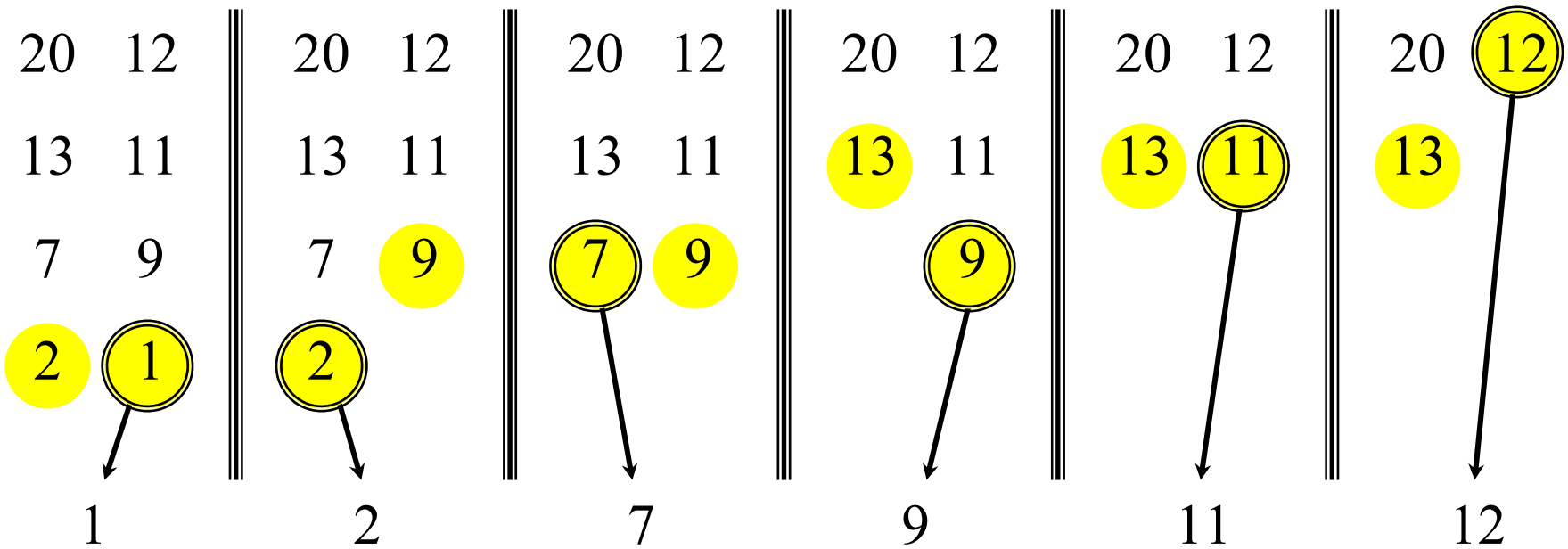
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



Time = $\Theta(n)$ to merge a total of n elements (linear time).

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

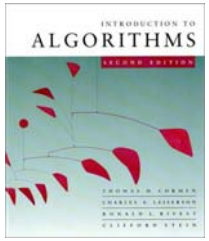
Analyzing merge sort

$T(n)$	**MERGE-SORT** $A[1 \,..\, n]$
$\Theta(1)$	1. If $n = 1$, done.
$2T(n/2)$	2. Recursively sort $A[\,1 \,..\, \lceil n/2 \rceil\,]$ and $A[\,\lceil n/2 \rceil + 1 \,..\, n\,]$.
$\Theta(n)$	***3. "Merge"*** the 2 sorted lists

Abuse

Sloppiness: Should be $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$, but it turns out not to matter asymptotically.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Recurrence for merge sort

$$T(n) = \begin{cases} \Theta(1) \text{ if } n = 1; \\ 2T(n/2) + \Theta(n) \text{ if } n > 1. \end{cases}$$

- We shall usually omit stating the base case when $T(n) = \Theta(1)$ for sufficiently small n, but only when it has no effect on the asymptotic solution to the recurrence.
- CLRS and Lecture 2 provide several ways to find a good upper bound on $T(n)$.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

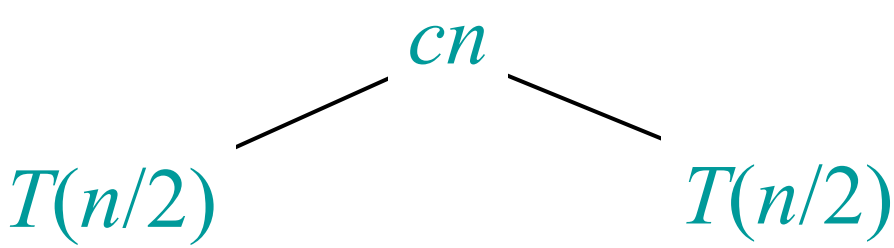
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

$$T(n)$$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

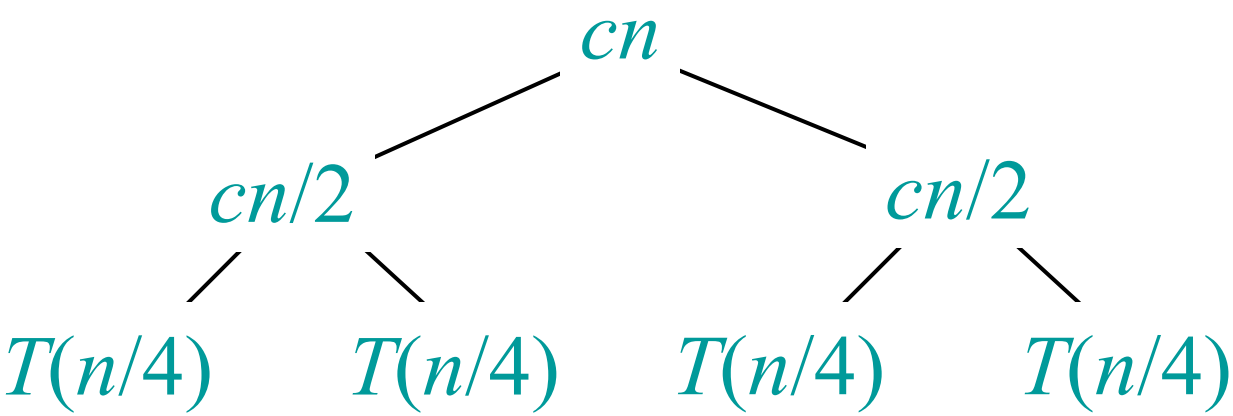
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

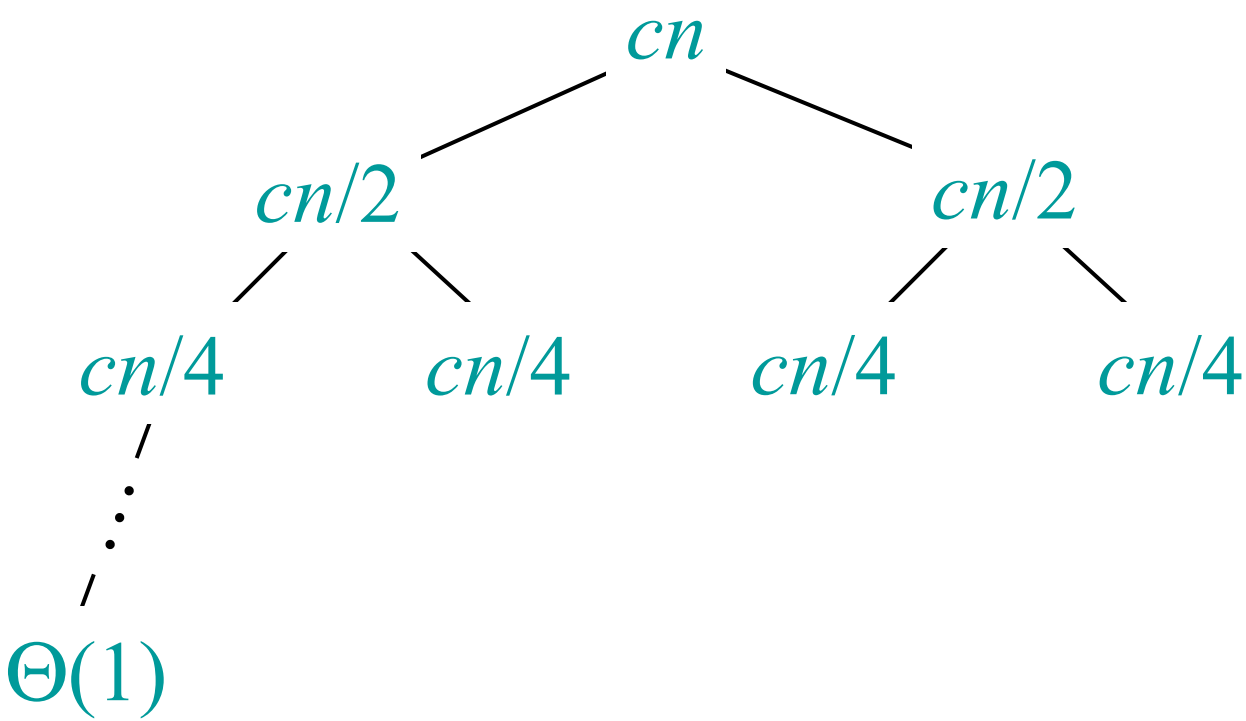
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

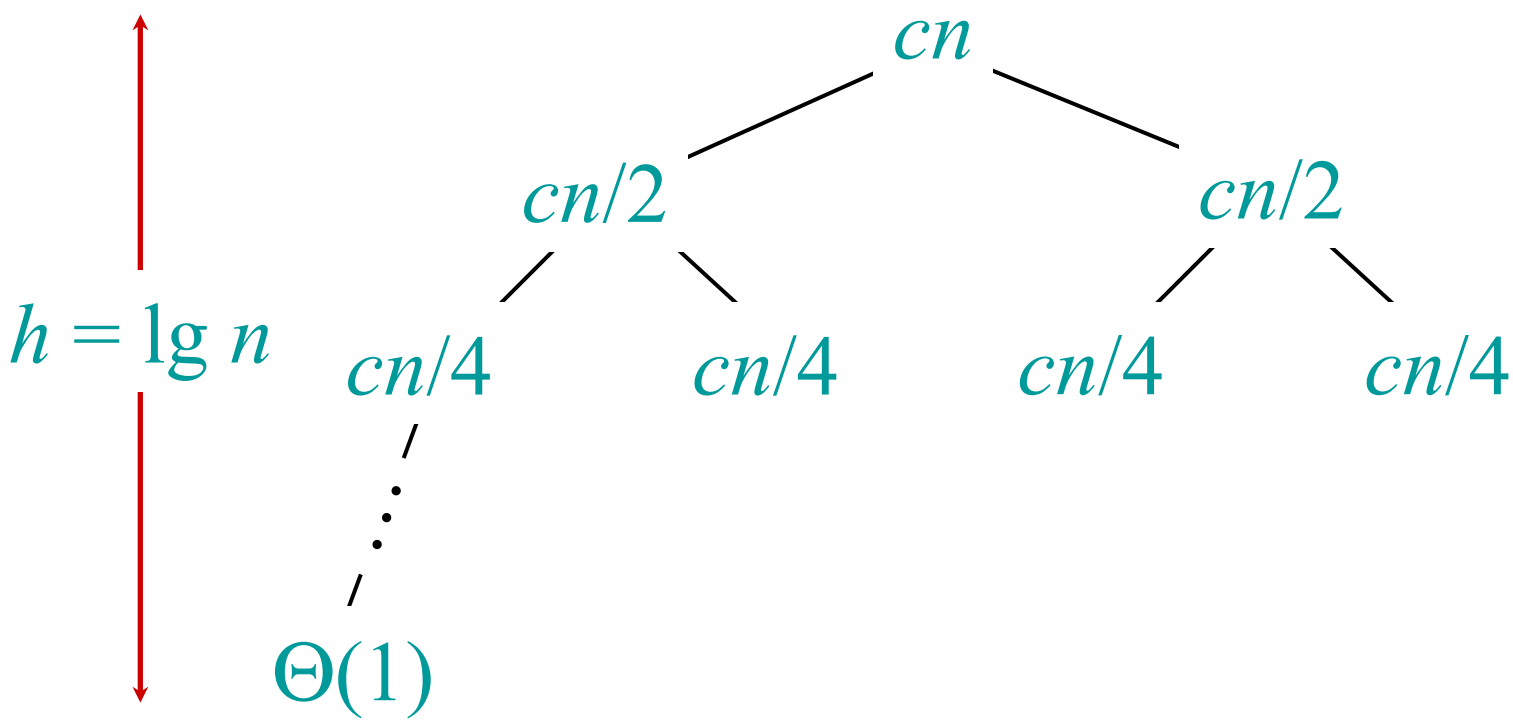
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

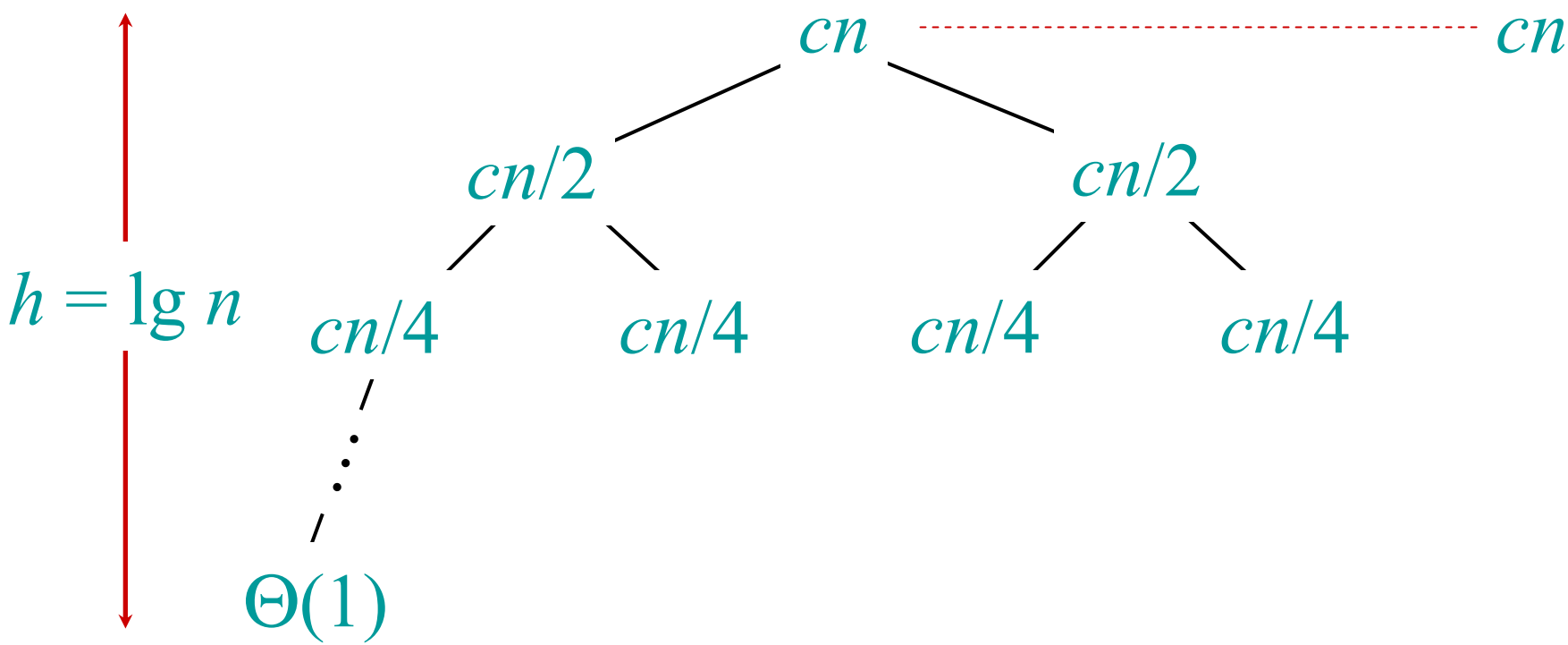
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

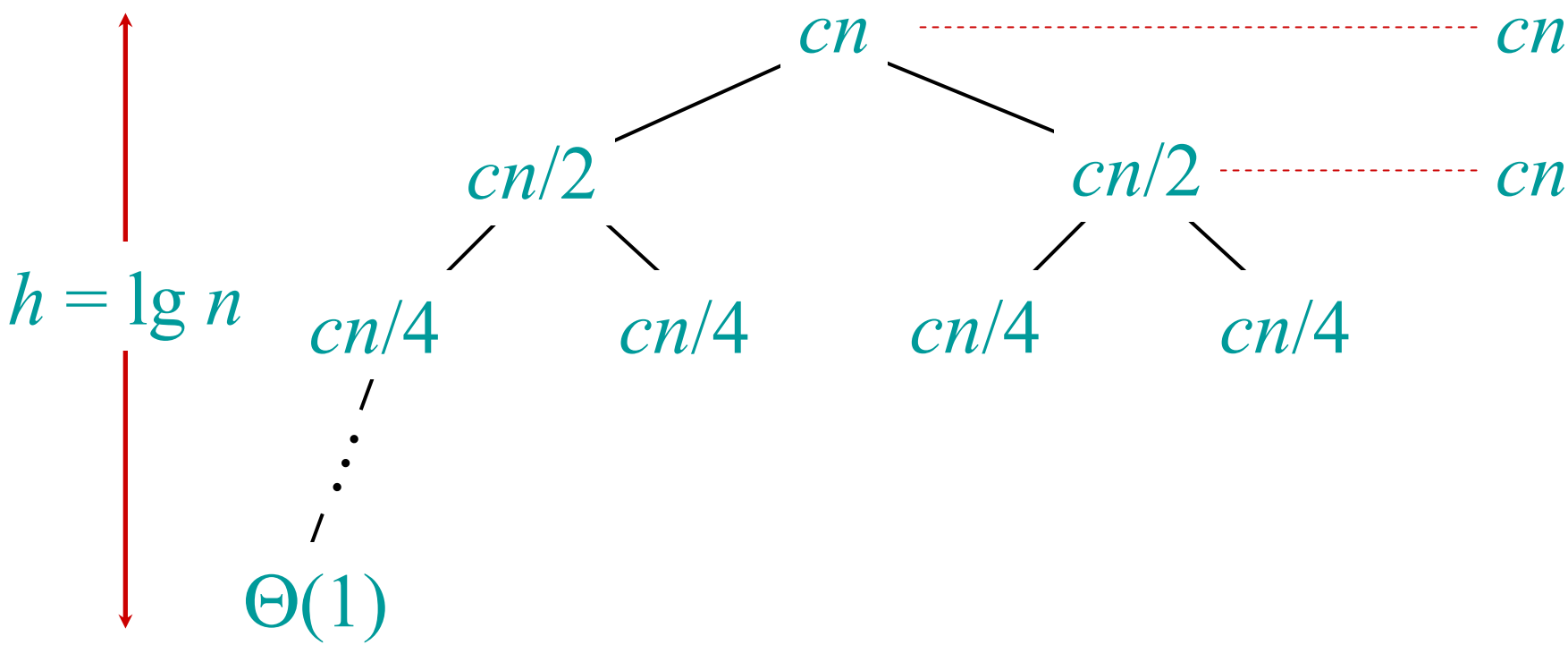
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

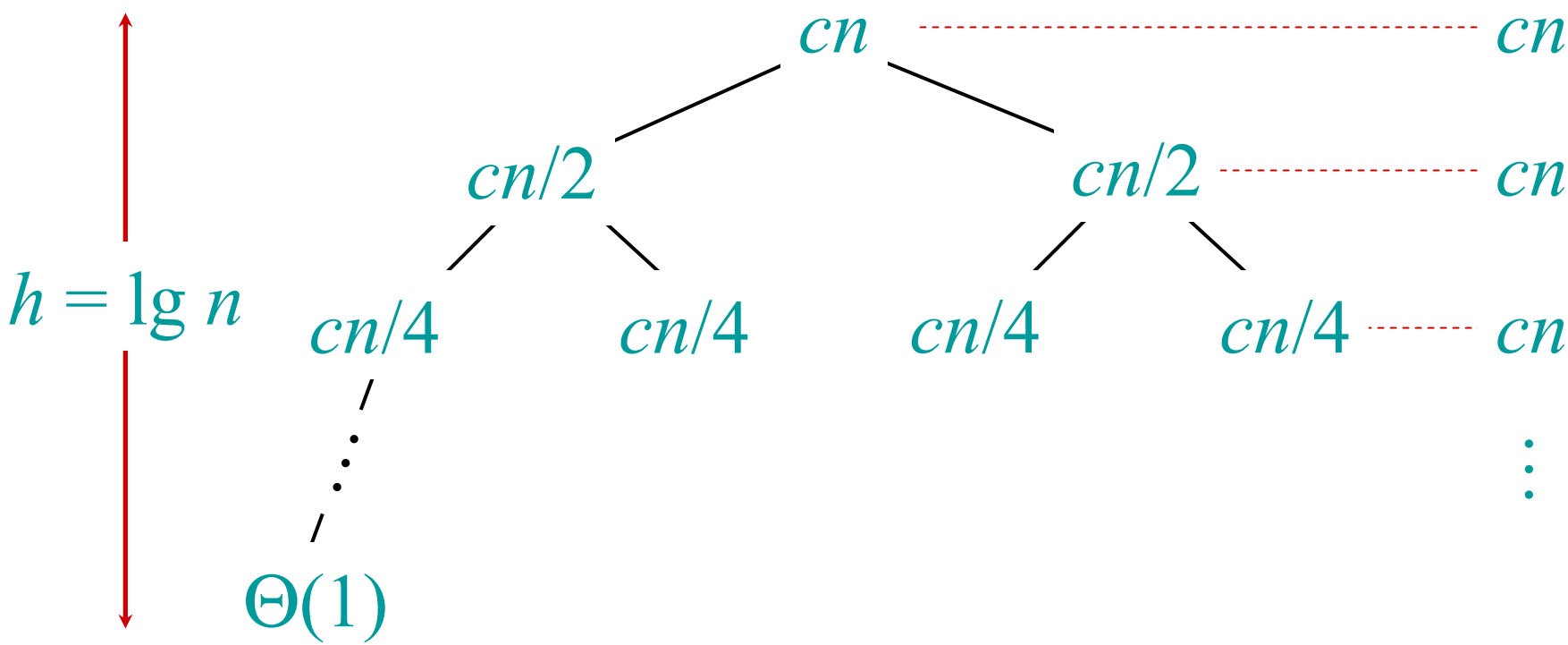
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

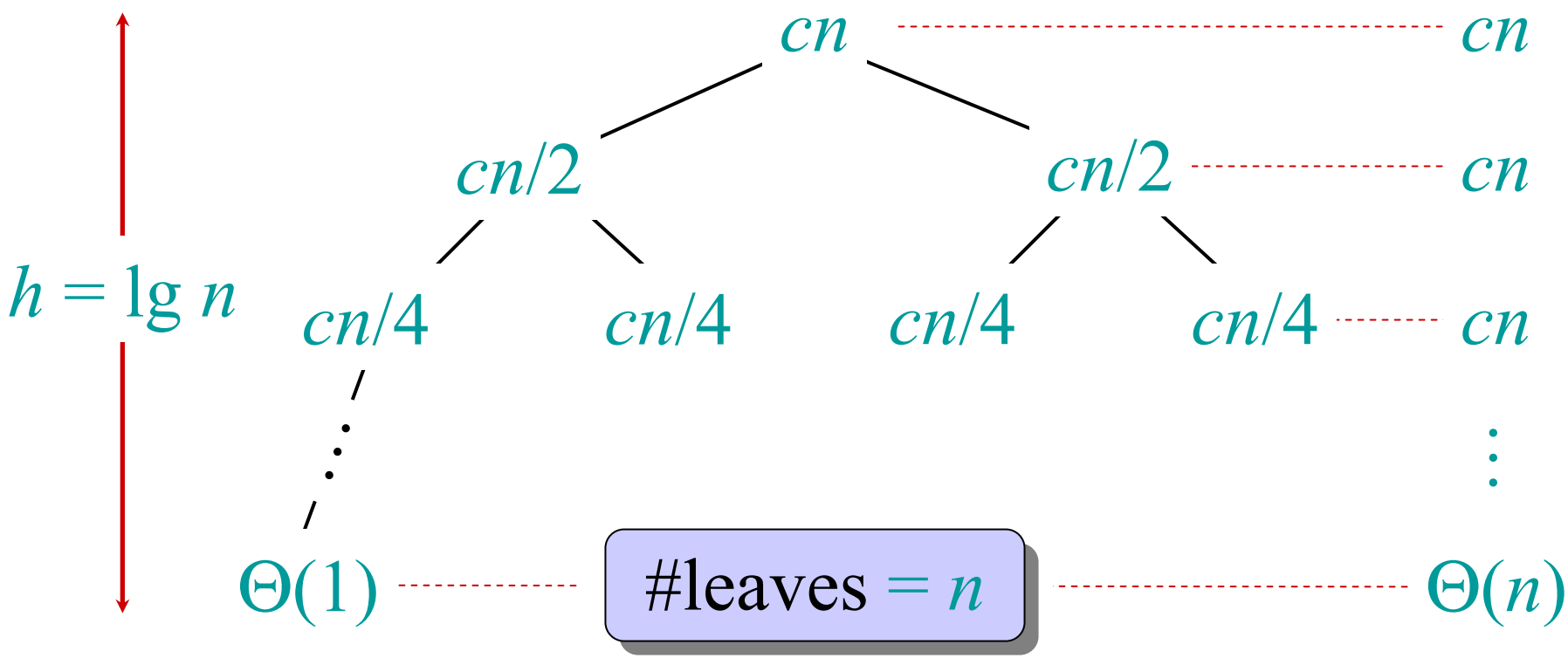
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

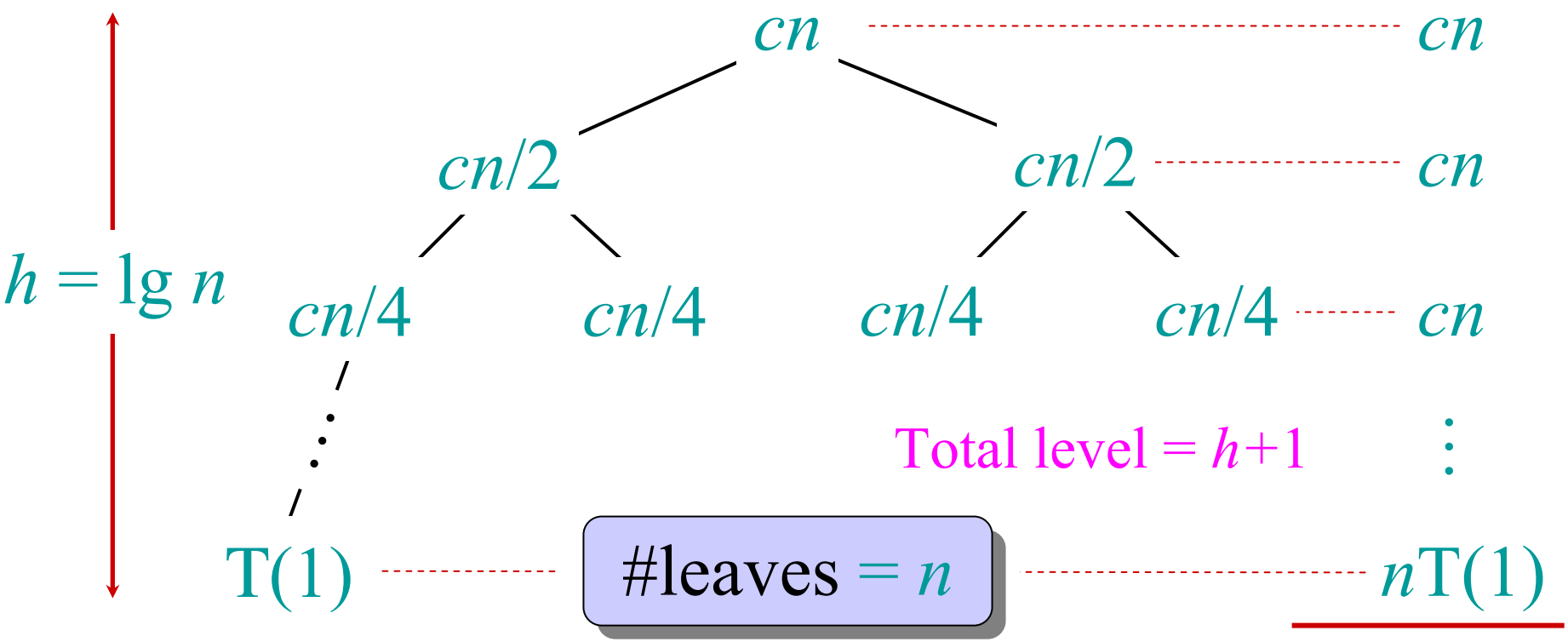
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

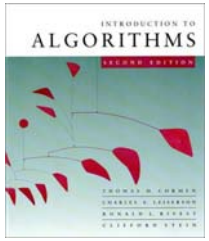
Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



$$T(n)=cn\lg n+nT(1)=\Theta(n \lg n)$$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Efficiency

Two computers:

A -- ten billion instructions per second, insert sort

B -- ten million instructions per second, merge sort

To sort ten million numbers

computer A takes

$$2(10^7)^2/10^{10} = 20000 \text{ s}$$

computer B takes

$$50(10^7 \lg 10^7) / 10^7 = 1163 \text{ s}$$

Conclusions

- $\Theta(n \lg n)$ grows more slowly than $\Theta(n^2)$.
- Therefore, merge sort asymptotically beats insertion sort in the worst case.
- In practice, merge sort beats insertion sort for $n > 30$ or so.
- Go test it out for yourself!

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson